ONBRD-08: Your First Queries

> **Series:** ONBRD — Dynatrace Onboarding | **Notebook:** 8 of 10 | **Created:** December 2025 | **Last Updated:** 05/06/2026

Learning DQL Fundamentals

Dynatrace Query Language (DQL) is how you access data in Grail. This notebook introduces the core concepts and patterns you'll use daily.

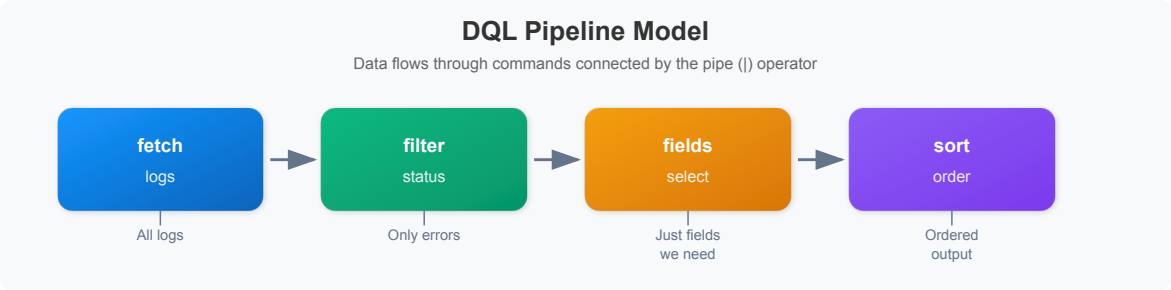
Table of Contents

Prerequisites

- Dynatrace environment with data (ONBRD-05, ONBRD-06, ONBRD-07)
- DQL query permissions
- Access to Notebooks or the DQL query interface

1. DQL Basics

DQL is a **pipeline-based query language**—not SQL. Data flows through a series of commands connected by the pipe (`|`) operator.



DQL vs SQL

DQL	SQL	Note
`fetch logs`	`SELECT * FROM logs`	Start with data source
`| filter x == "y"`	`WHERE x = 'y'`	Use `==`, double quotes
`| fields a, b`	`SELECT a, b`	Field selection after fetch
`| summarize count()`	`SELECT COUNT(*)`	Aggregation command
`by: {field}`	`GROUP BY field`	Grouping syntax
`{"a", "b"}`	`('a', 'b')`	Array syntax (curly braces)

2. The Pipeline Model

Each command in the pipeline operates on the output of the previous command:

```
fetch logs                          // 1. Get all logs
| filter loglevel == "error"      // 2. Keep only errors
| filter timestamp &gt; now() - 1h  // 3. Last hour only
| fields timestamp, content       // 4. Select columns
| sort timestamp desc             // 5. Order by time
| limit 100                       // 6. Take first 100
```

Order Matters

- **Filter early** - Reduces data before expensive operations
- **Select fields** - Reduces memory usage
- **Aggregate** - Summarize before sorting
- **Sort** - Order the final results
- **Limit** - Control output size

3. Fetching Data

Every DQL query starts with a `fetch` command specifying the data source.

```
// Fetch logs (most common)
fetch logs, from:-1h
| limit 10
```

```
// Fetch spans (distributed traces)
fetch spans, from:-1h
| limit 10
```

```
// Fetch entity data (hosts)
fetch dt.entity.host
| limit 10

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | limit 10
```

```
// Fetch problems
fetch dt.davis.problems, from:-24h
| limit 10
```

Common Data Sources

Source	Description
`logs`	Log records

Source	Description
`spans`	Distributed trace spans
`events`	System events
`bizevents`	Business events
`dt.entity.host`	Host entities
`dt.entity.service`	Service entities
`dt.entity.process_group`	Process group entities
`dt.davis.problems`	Detected problems

4. Filtering

Use `filter` to narrow results. Filter as early as possible for performance.

```
// Filter by equality
fetch logs, from:-1h
| filter loglevel == "error"
| limit 20
```

```
// Filter with multiple conditions (AND)
fetch logs, from:-1h
| filter loglevel == "error"
| filter timestamp &gt; now() - 1h
| limit 20
```

```
// Filter with or condition
fetch logs, from:-1h
| filter loglevel == "error" or loglevel == "warn"
| limit 20
```

```
// Filter using IN for multiple values
fetch logs, from:-1h
| filter in(loglevel, {"error", "warn", "fatal"})
| limit 20
```

```
// Filter with string matching
fetch logs, from:-1h
| filter contains(content, "timeout")
| limit 20
```

Filter Operators

Operator	Example	Description
`==`	`field == "value"`	Equals
`!=`	`field != "value"`	Not equals
`>`, `<`	`count > 10`	Greater/less than
`>=`, `<=`	`count >= 10`	Greater/less or equal
`and`, `or`	`a == 1 and b == 2`	Logical operators
`in()`	`in(field, {"a", "b"})`	Value in set
`contains()`	`contains(field, "text")`	Substring match
`isNull()`	`isNull(field)`	Field is null
`isNotNull()`	`isNotNull(field)`	Field is not null

5. Selecting Fields

Use `fields` to select specific columns. This improves readability and performance.

```
// Select specific fields from logs
fetch logs, from:-1h
| fields timestamp, loglevel, log.source, content
| limit 20
```

```
// Create calculated fields (duration / 1ms uses duration arithmetic —
preferred)
fetch spans, from:-1h
| fields span.name,
         duration,
         duration_ms = duration / 1ms
| limit 20
```

```
// Rename fields with aliases
fetch dt.entity.host
| fields name = entity.name,
         status = state,
         os = osType
| limit 20
```

fieldsAdd vs fields

Command	Effect
`fields`	Keeps only specified fields
`fieldsAdd`	Adds new fields, keeps all existing

```
// fieldsAdd keeps existing fields and adds new ones
fetch spans, from:-1h
| fieldsAdd duration_ms = duration / 1ms
| fields span.name, duration, duration_ms
| limit 10
```

6. Aggregating with Summarize

Use `summarize` to aggregate data. Combine with `by:` for grouping.

```
// Simple count
fetch logs, from: now() - 1h
| summarize total_logs = count()
```

```
// Count by group
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {loglevel}
| sort log_count desc
```

```
// Multiple aggregations
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize
    request_count = count(),
    avg_duration = avg(duration),
    max_duration = max(duration),
    by: {service.name}
| sort request_count desc
| limit 20
```

```
// Conditional counting
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize
    total = count(),
    errors = countIf(span.status_code == "error"),
    by: {service.name}
| fieldsAdd error_rate = 100.0 * errors / total
| sort error_rate desc
| limit 20
```

Common Aggregation Functions

Function	Description
`count()`	Count records
`countIf(condition)`	Count where condition is true
`sum(field)`	Sum values
`avg(field)`	Average value
`min(field)`	Minimum value
`max(field)`	Maximum value
`percentile(field, 95)`	95th percentile

7. Sorting and Limiting

Control output order and size.

```
// Sort descending (newest first)
fetch logs, from: now() - 1h
| fields timestamp, loglevel, content
| sort timestamp desc
| limit 20
```

```
// Sort by multiple fields
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize request_count = count(), by: {service.name}
| sort request_count desc
| limit 10
```

```
// Find slowest spans
fetch spans, from: now() - 1h
| fields span.name, service.name, duration
| sort duration desc
| limit 10
```

8. Time Ranges

Control the time range for your queries.

```
// Last hour (using the from: parameter on fetch)
fetch logs, from: now() - 1h
| summarize count()
```

```
// Specific time range
fetch logs, from: now() - 24h, to: now() - 12h
| summarize count()
```

```
// Last 7 days
fetch dt.davis.problems, from: now() - 7d
| summarize problem_count = count(), by: {event.status}
```

Time Units

Unit	Example	Description
`s`	`now() - 30s`	Seconds
`m`	`now() - 15m`	Minutes
`h`	`now() - 2h`	Hours
`d`	`now() - 7d`	Days

9. Common Patterns

Here are patterns you'll use frequently.

Error Investigation

```
// Find error logs with context
fetch logs, from: now() - 1h
| filter loglevel == "error"
| fields timestamp, log.source, content
| sort timestamp desc
| limit 50
```

Service Performance

```
// Service response time summary (use duration / 1ms for ns→ms conversion)
fetch spans, from: now() - 1h
| filter span.kind == "server"
| summarize {
    requests = count(),
    avg_ms = avg(duration) / 1ms,
    p95_ms = percentile(duration, 95) / 1ms
  }, by: {service.name}
| sort requests desc
| limit 20
```

Log Volume Analysis

```
// Log volume by source and severity
fetch logs, from: now() - 1h
| summarize log_count = count(), by: {log.source, loglevel}
| sort log_count desc
| limit 30
```

Entity Inventory

```
// Host inventory with details
fetch dt.entity.host
| fields
    name = entity.name,
    state,
    os = osType,
    cores = cpuCores
| sort name
| limit 50
```

10. Next Steps

With DQL fundamentals covered:

1. **ONBRD-09: Setting Up Alerts** — Configure alerting and notifications
2. Practice with your own data
3. Explore the DQL documentation for advanced functions
4. Try creating queries in Notebooks

Beyond the Basics

- **`makeTimeseries`** — convert event-based data (logs, spans, bizevents) into time-bucketed metric series. Different from the `timeseries` command, which queries pre-ingested metrics. Example: `fetch logs | makeTimeseries error_rate = countIf(loglevel == "ERROR"), interval:5m, by:{k8s.cluster.name}`
- **Iterative array expressions** — `arrayAvg`, `arraySum`, `iAny`, etc. for working with timeseries arrays in `fieldsAdd` and `filter`

- **Smartscape topology navigation** — `smartscapeNodes`, `smartscapeEdges`, `traverse` for entity relationships (the modern alternative to `dt.entity.*`)

Where to Go Deeper

- **OPLOGS / OPMIG / OPIPE** — Log query patterns, OpenPipeline transformations
- **SPANS series** — Span-specific DQL patterns
- **DASH series** — Building dashboards from DQL queries
- **AIOPS series** — Davis problem queries, anomaly detection DQL

DQL Checklist

- [ ] Understand the pipeline model
- [ ] Can fetch different data types
- [ ] Can filter with conditions
- [ ] Can select and calculate fields (using duration arithmetic, not nanosecond constants)
- [ ] Can aggregate with summarize
- [ ] Can sort and limit results
- [ ] Can specify time ranges
- [ ] Know that `makeTimeseries` exists for event → series conversion

Summary

In this notebook, you learned:

- DQL uses a pipeline model (not SQL)
- `fetch` starts every query
- `filter` narrows results
- `fields` selects and calculates columns
- `summarize` aggregates data
- `sort` and `limit` control output
- Time ranges with `from:` and `to:`
- Duration arithmetic (`duration / 1ms`) is preferred over nanosecond constants

References

- DQL Reference
- DQL Functions
- DQL Commands
- DQL Examples

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.